

# Employee Entity 設計方針書

プロジェクト: SupportAdvance

レイヤ: Domain (Employee Bounded Context)

対象: Employee Entity の根本的な設計方針

版: 1.0 / 2026-08-09

## 1. 基本設計方針

### 1.1 Employee Entity の位置づけ

- 集約根:** Employee Bounded Context のドメインモデル
- 責務:** 従業員情報の管理と一貫性の確保
- 参照方式:** 複数の BC から参照される (Pattern 3: ID のみ共有)

### 1.2 Bounded Context 間の参照戦略

採用パターン: Pattern 3 - EmployeeCode で共有 (最も疎結合で業務的意味を持つ)

```
各 Bounded Context は EmployeeCode を保持
├─ Employee BC (データ所有者)
│   └─ m_employees テーブル (完全なドメインモデル)
├─ HumanResources BC
│   └─ Salary Entity (EmployeeCode を保持、給与計算・検索に使用)
├─ CarPreferences BC
│   └─ CarPreference Entity (EmployeeCode を保持、監査ログで「M1234 が作成」と記録)
└─ Authentication BC
    └─ EmployeeCode で認証 (ログイン入力値として M1234 を受け取る)
```

理由:

- ✓ BC の独立性が保証される
- ✓ EmployeeCode は人間が読める (PersonId よりビジネス的意味がある)
- ✓ 各 BC で直接的に従業員を識別 (余計な DB 問い合わせ不要)
- ✓ Employee BC はデータ所有者として一元管理
- ✓ EmployeeCode は一意 (UNIQUE 制約)
- ✓ 監査ログで「誰が」が直感的にわかる

EmployeeCode は ValueObject として全 BC で共有され、詳細情報が必要な場合は Employee BC が提供する Application Service を呼び出す。

## 2. DB テーブル設計

### 2.1 マスタテーブル

#### m\_employees (従業員マスタ)

```
CREATE TABLE [dbo].[m_employees]
(
    -- 監査カラム (すべてのテーブルで必須)
    [row_id] [bigint] NOT NULL PRIMARY KEY DEFAULT (NEXT VALUE FOR [dbo].[s_row_id_sequence]),
    [row_version] [timestamp] NOT NULL,
    [created_at] [datetime2](7) NOT NULL,
    [created_by] [bigint] NOT NULL,
    [updated_at] [datetime2](7) NULL,
    [updated_by] [bigint] NULL,
    [deleted_at] [datetime2](7) NULL,
    [deleted_by] [bigint] NULL,

    -- ビジネスカラム
    [employee_agg_id] [uniqueidentifier] NOT NULL UNIQUE, -- 集約ID (GUID)
    [employee_id] [int] NOT NULL UNIQUE, -- 従業員ID (1001～)

    -- 制約
    CONSTRAINT [FK_mEmployees_CreatedBy] FOREIGN KEY ([created_by]) REFERENCES [m_persons]([row_id]),
    CONSTRAINT [FK_mEmployees_UpdatedBy] FOREIGN KEY ([updated_by]) REFERENCES [m_persons]([row_id]),
    CONSTRAINT [FK_mEmployees_DeletedBy] FOREIGN KEY ([deleted_by]) REFERENCES [m_persons]([row_id])
)
```

#### m\_roles (ロールマスタ)

```
CREATE TABLE [dbo].[m_roles]
(
    [row_id] [bigint] NOT NULL PRIMARY KEY DEFAULT (NEXT VALUE FOR [dbo].[s_row_id_sequence]),
    [row_version] [timestamp] NOT NULL,
    [role_id] [int] NOT NULL UNIQUE, -- 業務ID (Admin=10, Manager=20 など)
    [role_name] [nvarchar](100) NOT NULL, -- 表示名 (Admin など)
    [created_at] [datetime2](7) NOT NULL,
    [created_by] [bigint] NOT NULL,
    [updated_at] [datetime2](7) NULL,
    [updated_by] [bigint] NULL,
    [deleted_at] [datetime2](7) NULL,
)
```

```

    [deleted_by] [bigint] NULL
)

```

## m\_permissions (権限マスタ)

```

CREATE TABLE [dbo].[m_permissions]
(
    [row_id] [bigint] NOT NULL PRIMARY KEY DEFAULT (NEXT VALUE FOR [dbo].[s_row_id_sequence])
    [row_version] [timestamp] NOT NULL,
    [permission_id] [int] NOT NULL UNIQUE,      -- 業務ID (Create=1, Read=2 など)
    [permission_name] [nvarchar](100) NOT NULL, -- 表示名 (Create など)
    [created_at] [datetime2](7) NOT NULL,
    [created_by] [bigint] NOT NULL,
    [updated_at] [datetime2](7) NULL,
    [updated_by] [bigint] NULL,
    [deleted_at] [datetime2](7) NULL,
    [deleted_by] [bigint] NULL
)

```

## 2.2 トランザクションテーブル

### t\_employee\_attributes (従業員属性)

```

CREATE TABLE [dbo].[t_employee_attributes]
(
    [row_id] [bigint] NOT NULL PRIMARY KEY DEFAULT (NEXT VALUE FOR [dbo].[s_row_id_sequence])
    [row_version] [timestamp] NOT NULL,
    [employee_row_id] [bigint] NOT NULL,      -- FK to m_employees
    [attribute_type] [nvarchar](50) NOT NULL, -- 'Person', 'Role', 'Permission', 'Certification'
    [attribute_row_id] [bigint] NOT NULL,     -- FK to m_persons or m_roles or m_permissions
    [created_at] [datetime2](7) NOT NULL,
    [created_by] [bigint] NOT NULL,
    [updated_at] [datetime2](7) NULL,
    [updated_by] [bigint] NULL,
    [deleted_at] [datetime2](7) NULL,
    [deleted_by] [bigint] NULL,

    CONSTRAINT [FK_tEmployeeAttributes_EmployeeRowId] FOREIGN KEY ([employee_row_id]) REFERENCES [m_employees] ([row_id])
    CONSTRAINT [FK_tEmployeeAttributes_CreatedBy] FOREIGN KEY ([created_by]) REFERENCES [m_employees] ([row_id])
)

```

#### 特徴:

- 単一テーブルで Person、Role、Permission、Certification など複数の属性タイプを管理

- 新しい属性タイプ追加時はスキーマ変更不要（行追加のみ）
- attribute\_type で柔軟に拡張可能
- Employee Entity は Person情報を保持しない（t\_employee\_attributes で管理）

---

## 3. ValueObject 設計ルール

---

### 3.1 統一ルール

すべての業務ID は ValueObject でラップ

パターン1：単純な業務ID

```
// Domain層 (Entity/ValueObject)
public sealed class PersonId : PrimitiveValueObject<int> { }
public sealed class RoleId : PrimitiveValueObject<int> { }
public sealed class PermissionId : PrimitiveValueObject<int> { }

// DbModel層 (Infrastructure)
public class EmployeeDbModel
{
    public Guid EmployeeAggId { get; set; } // プリミティブ型 (GUID)
    public char EmployeeDivision { get; set; } // プリミティブ型 (M/T/C)
    public int EmployeeId { get; set; } // プリミティブ型 (従業員ID)
    public int PersonId { get; set; } // プリミティブ型
    public int RoleId { get; set; } // プリミティブ型
    public int PermissionId { get; set; } // プリミティブ型
}

// Mapper層
public EmployeeAggId ToDomainAggId(Guid dbValue) => EmployeeAggId.From(dbValue);
public Guid ToDbAggId(EmployeeAggId domainValue) => domainValue.Value;

public EmployeeDivision ToDomainDivision(char dbValue) => EmployeeDivision.From(dbValue);
public char ToDbDivision(EmployeeDivision domainValue) => domainValue.Value;

public EmployeeId ToDomainEmployeeId(int dbValue) => EmployeeId.From(dbValue);
public int ToDbEmployeeId(EmployeeId domainValue) => domainValue.Value;

public PersonId ToDomainPersonId(int dbValue) => PersonId.From(dbValue);
public int ToDbPersonId(PersonId domainValue) => domainValue.Value;
```

## パターン2：複合値ID

```
// Domain層
public sealed class EmployeeCode : ValueObject
{
    public EmployeeDivision Division { get; private set; }
    public EmployeeId Id { get; private set; }
    // M1234 形式で表示
}

// DbModel層
public class EmployeeDbModel
{
    public char EmployeeDivision { get; set; } // プリミティブ型
    public int EmployeeId { get; set; } // プリミティブ型
}

// Mapper層
public EmployeeCode ToEmployeeCode(char division, int employeeId)
=> EmployeeCode.From(
    EmployeeDivision.From(division),
    EmployeeId.From(employeeId));
```

### 3.2 型の対応

| Domain層 (Entity/ValueObject)   | Infrastructure層 (DbModel)               |
|--------------------------------|-----------------------------------------|
| EmployeeAggId (ValueObject)    | uniqueidentifier employee_agg_id        |
| EmployeeDivision (ValueObject) | char employee_division                  |
| EmployeeId (ValueObject)       | int employee_id                         |
| EmployeeCode (複合ValueObject)   | char employee_division, int employee_id |
| PersonId (ValueObject)         | int person_id                           |
| RoleId (ValueObject)           | int role_id                             |
| PermissionId (ValueObject)     | int permission_id                       |
| LocalDateTime                  | datetime2                               |

## 4. Employee Entity 設計

### 4.1 Entity 構造

```
public sealed class Employee : Entity<EmployeeAggId>
{
    // 公開プロパティ (他の層からアクセス可能)
    public EmployeeId EmployeeId { get; private set; } // 従業員ID (1001以上)
    public EmployeeCode EmployeeCode { get; private set; } // 複合コード (M1234)、BC間で共有

    // 内部用プロパティ (Repository のみでアクセス)
    internal EmployeeRowId RowId { get; private set; } // 初期値0L、Insert後に生成
    internal LocalDateTime CreatedAt { get; private set; }
    internal long CreatedBy { get; private set; }
    internal LocalDateTime? UpdatedAt { get; private set; }
    internal long? UpdatedBy { get; private set; }
    internal LocalDateTime? DeletedAt { get; private set; }
    internal long? DeletedBy { get; private set; }
}
```

### 4.2 キー設計の理由

| キー            | 型                     | プロパティ名       | 役割                            | 公開度 |
|---------------|-----------------------|--------------|-------------------------------|-----|
| EmployeeAggId | Guid<br>(ValueObject) | base.Id      | 集約根ID (GUID)、Entityから継承       | 公開  |
| EmployeeId    | ValueObject           | EmployeeId   | 従業員ID (1001以上)                | 公開  |
| EmployeeCode  | 複合<br>ValueObject     | EmployeeCode | 従業員コード (M1234)、BC間で共有         | 公開  |
| EmployeeRowId | long                  | RowId        | DB行ID<br>(m_employees.row_id) | 内部用 |

注: PersonId、PersonRowId、EmployeeDivision は t\_employee\_attributes で管理

### 4.3 属性データ (Person/Role/Permission) の扱い

Employee Entity は Person、Roles、Permissions を保持しない

理由：

- ☒ Entity がシンプル（ビジネスID のみ保持）
- ☒ BC 内での疎結合
- ☒ 各属性テーブル（m\_persons、m\_roles、m\_permissions）の独立性を保証
- ☒ t\_employee\_attributes で属性を柔軟に管理

```
// ☒ 保持しない
public List<EmployeeRole> Roles { get; private set; }
public PersonId PersonId { get; private set; }

// ☒ Repository で取得
public class EmployeeRepository
{
    public async Task<PersonInfo> GetPersonInfoAsync(EmployeeAggId aggId)
    {
        // t_employee_attributes から attribute_type='Person' の行を取得
    }

    public async Task<List<EmployeeRole>> GetRolesByEmployeeAsync(EmployeeAggId aggId)
    {
        // t_employee_attributes から attribute_type='Role' の行を取得
    }

    public async Task<List<EmployeePermission>> GetPermissionsByEmployeeAsync(EmployeeAggId
    {
        // t_employee_attributes から attribute_type='Permission' の行を取得
    }
}
```

## 5. Insert フロー

### 5.1 Application層（Use Case）

```
// 1. Employee Entity を生成 (RowId=0L で初期化)
var division = EmployeeDivision.From(divisionChar);
var employeeId = EmployeeId.From(employeeIdInt);
var code = EmployeeCode.From(division, employeeId);

var employee = Employee.Create(
    EmployeeAggId.NewId(),
    EmployeeRowId.From(0L),           // 未セット (DB で生成)
    employeeId,
```

```

        code
    );

    // Person 情報は t_employee_attributes で別途登録

    // 2. Repository に保存
    await _employeeRepository.AddAsync(employee);

```

## 5.2 Repository層

```

public async Task AddAsync(Employee entity)
{
    var dbModel = MapToDatabaseForInsert(entity);

    // ストアドプロシージャ実行 (Insert + SCOPE_IDENTITY())
    var generatedRowId = await _dataAccess.InsertAndReturnRowIdAsync(dbModel);

    // 生成された RowId で Entity を再作成
    var createdEmployee = Employee.Reconstruct(
        entity.Id, // AggId (Entity<TId>.Id)
        EmployeeRowId.From(generatedRowId), // DB で生成された RowId
        entity.EmployeeId, // ID を引き継ぎ
        entity.EmployeeCode // Code を引き継ぎ
    );
}

```

## 5.3 ストアドプロシージャ (SQL Server)

```

CREATE PROCEDURE [dbo].[sp_InsertEmployee]
    @EmployeeAggId UNIQUEIDENTIFIER,
    @EmployeeDivision CHAR(1),
    @EmployeeId INT,
    @PersonId INT,
    @PersonRowId BIGINT,
    @CreatedAt DATETIME2(7),
    @CreatedBy BIGINT
AS
BEGIN
    INSERT INTO [dbo].[m_employees]
    (employee_agg_id, employee_division, employee_id, person_id, person_row_id, created_at,
    VALUES
    (@EmployeeAggId, @EmployeeDivision, @EmployeeId, @PersonId, @PersonRowId, @CreatedAt, @C

    -- 生成された RowId と監査情報を返す
    OUTPUT INSERTED.row_id, INSERTED.row_version, INSERTED.employee_division, INSERTED.emplc

```

```
INSERTED.created_at, INSERTED.created_by;
```

```
END
```

## 6. 関連ドメインモデル

### 6.1 EmployeeDivision ValueObject

- 従業員区分（M=従業員、T=派遣社員、C=請負者）
- EmployeeCode に含まれる

### 6.2 EmployeeId ValueObject

- 従業員ID（1001以上、無上限）
- EmployeeCode に含まれる

### 6.3 EmployeeCode ValueObject

- 複合値（Division + EmployeeId）
- 表示形式：M1234
- 範囲検証を内部で実施

### 6.4 Person マスタ

- 人物マスタ（m\_persons）
- PersonId（業務ID） + PersonRowId（DB行ID）で参照

## 7. 監査カラムの責務

すべてのテーブルは8つの監査カラムを必須とする

| カラム         | 作成時           | 更新時           | 削除時  | 責務      |
|-------------|---------------|---------------|------|---------|
| row_id      | 自動採番          | 変更なし          | 変更なし | DB技術的ID |
| row_version | 自動生成          | 自動更新          | 自動更新 | 楽観ロック   |
| created_at  | Application設定 | 変更なし          | 変更なし | 作成日時    |
| created_by  | Application設定 | 変更なし          | 変更なし | 作成者     |
| updated_at  | NULL          | Application設定 | 変更なし | 更新日時    |

| カラム        | 作成時  | 更新時           | 削除時           | 責務     |
|------------|------|---------------|---------------|--------|
| updated_by | NULL | Application設定 | 変更なし          | 更新者    |
| deleted_at | NULL | 変更なし          | Application設定 | 論理削除日時 |
| deleted_by | NULL | 変更なし          | Application設定 | 削除者    |

---

## 8. 参考資料

- CLAUDE.md（プロジェクト全体の設計指針）
- TABLE\_DESIGN\_STANDARDS.md（DB設計標準）
- Repository\_パターンガイド.md（Repository実装ガイド）
- Aggregateld\_設計ガイド.md（集約根ID設計）